Sistema de Inspeção Visual Automatizada para Controle de Qualidade em Peças Fabricadas

***Israel Lucas*[1]**

[1]*Escola de Ciências e Tecnologia, UFRN,*
israel.lucas.126@ufrn.edu.br

Resumo

Este trabalho apresenta o desenvolvimento de um sistema de inspeção visual automatizada para detecção de defeitos em peças metálicas utilizando técnicas de visão computacional e aprendizagem profunda. O experimento foi desenvolvido no Google Colab com a biblioteca Ultralytics e o modelo YOLOv8n. O conjunto de dados utilizado, denominado Detection_Defects_in_Metal, possuía anotações originalmente no formato Pascal VOC (XML), que foram convertidas para o formato YOLO (TXT), com normalização das coordenadas das caixas delimitadoras. O conjunto foi dividido em 80% para treinamento e 20% para validação, resultando em 1.824 imagens de treinamento e 456 imagens de validação. O modelo foi treinado durante 10 épocas, com imagens de 640 × 640 pixels e batch size 16. Na validação final, foram obtidos precisão de 0,674, recall de 0,470, mAP@50 de 0,497 e mAP@50–95 de 0,247. Os resultados demonstram a viabilidade da aplicação do YOLOv8 na identificação automatizada de defeitos, embora também indiquem limitações relacionadas à distribuição desigual das classes e ao número reduzido de exemplos de algumas categorias.

Abstract

This work presents the development of an automated visual inspection system for detecting defects in metallic parts using computer vision and deep learning techniques. The experiment was developed in Google Colab using the Ultralytics library and the YOLOv8n model. The dataset, named Detection_Defects_in_Metal, originally contained Pascal VOC (XML) annotations, which were converted to the YOLO (TXT) format, with normalized bounding-box coordinates. The dataset was split into 80% for training and 20% for validation, resulting in 1,824 training images and 456 validation images. The model was trained for 10 epochs using 640 × 640 pixel images and a batch size of 16. In the final validation, the model achieved a precision of 0.674, recall of 0.470, mAP@50 of 0.497, and mAP@50–95 of 0.247. The results demonstrate the feasibility of applying YOLOv8 to automated defect identification, while also indicating limitations associated with class imbalance and the small number of samples in some categories.

Keywords: computer vision; visual inspection; object detection; YOLOv8; metallic parts.

1 Introdução

A Visão Computacional é uma área da ciência da computação que permite aos sistemas computacionais interpretar e extrair informações de imagens digitais, possibilitando a identificação de objetos, padrões e

características visuais. Segundo Gonzalez e Woods (2010), o processamento digital de imagens utiliza técnicas como filtragem, segmentação e reconhecimento de padrões para obter informações relevantes. No contexto industrial, essas técnicas são aplicadas à inspeção visual automatizada, contribuindo para o controle de qualidade e a identificação de irregularidades em produtos fabricados.

Em peças metálicas, a detecção automatizada permite identificar diferentes tipos de imperfeições superficiais por meio da análise de imagens. Conforme Xie (2008), técnicas de análise de textura apresentam aplicações importantes na detecção de defeitos em superfícies industriais. Dessa forma, a utilização de sistemas computacionais pode auxiliar na padronização das inspeções e reduzir a dependência de avaliações exclusivamente manuais.

Com os avanços da Inteligência Artificial, as Redes Neurais Convolucionais (*CNNs*) passaram a desempenhar um papel importante na análise de imagens, permitindo aprender características como bordas, formas e texturas diretamente dos dados de treinamento (LECUN; BENGIO; HINTON, 2015). Entre os algoritmos de detecção de objetos destaca-se o YOLO (*You Only Look Once*), apresentado por Redmon et al. (2016), que realiza simultaneamente a classificação e a localização dos objetos em uma imagem. Neste trabalho, foi utilizado o modelo YOLOv8n, da biblioteca Ultralytics, para identificar e localizar defeitos em peças metálicas por meio de caixas delimitadoras (*bounding boxes*).

O treinamento desses modelos depende de imagens devidamente anotadas, contendo a identificação das classes e as coordenadas dos objetos. Para isso, as anotações podem ser armazenadas em diferentes formatos, como Pascal VOC (XML) e YOLO (TXT). O desempenho do modelo pode ser avaliado por métricas como precisão (*Precision*), revocação (*Recall*) e precisão média (mAP), que permitem analisar a capacidade de identificação e localização dos defeitos.

No cenário industrial, a inspeção de produtos é uma etapa importante para garantir a qualidade e a conformidade das peças fabricadas. Em processos com grande quantidade de produtos, a inspeção exclusivamente manual pode apresentar limitações relacionadas à repetibilidade, ao tempo de execução e à possibilidade de variações na avaliação realizada pelo operador. Nesse contexto, sistemas de inspeção visual automatizada podem auxiliar na identificação de características e defeitos presentes nas peças.

A visão computacional possibilita transformar imagens em informações úteis para sistemas automatizados. Entre as aplicações dessa área encontra-se a detecção de objetos, na qual o modelo deve localizar objetos ou regiões de interesse e atribuir uma classe a cada detecção. Para a inspeção de peças metálicas, essa abordagem pode ser utilizada para identificar diferentes tipos de imperfeições a partir de imagens.

2 Metodologia

A metodologia adotada neste trabalho consistiu no desenvolvimento de um sistema de inspeção visual automatizada para a detecção de defeitos em peças metálicas, utilizando técnicas de visão computacional e aprendizagem profunda. Para isso, foi utilizado um *dataset* encontrado no site do *Kaggle*. Nele é possível encontrar diversos compartilhamentos de *datasets*, bem como a possibilidade de criar um do zero. O experimento foi realizado no ambiente Google Colab, com a biblioteca Ultralytics e o modelo de detecção de objetos YOLOv8n.O procedimento foi dividido em três etapas principais: preparação do conjunto de dados, treinamento do modelo e avaliação dos resultados obtidos.

2.1 Conjunto de dados e preparação

O conjunto de dados utilizado no experimento é denominado Detection_Defects_in_Metal. As imagens estavam armazenadas em uma pasta de imagens e as respectivas anotações em arquivos XML. As anotações seguem a estrutura Pascal VOC, contendo, entre outras informações, as dimensões da imagem, o nome da classe e as coordenadas da caixa delimitadora (xmin, xmax, ymin e ymax).

Para permitir o treinamento com YOLO, os arquivos XML foram convertidos para arquivos TXT. As coordenadas das caixas foram transformadas para o formato YOLO, representado por classe, centro x, centro y, largura e altura, com os valores normalizados pelas dimensões da imagem. A divisão dos dados foi realizada com test_size=0,2 e random_state=42, resultando em 80% dos arquivos para treinamento e 20% para validação.

O processamento realizado no notebook gerou 1.824 imagens no conjunto de treinamento e 456 imagens no conjunto de validação. O treinamento utilizou 12 classes de defeitos presentes nas anotações.

2.2 Modelo e treinamento

Foi utilizado o modelo YOLOv8n pré-treinado, carregado por meio da biblioteca Ultralytics. O treinamento foi configurado para 10 épocas, tamanho de imagem de 640 pixels e batch size igual a 16. O treinamento foi executado no ambiente Google Colab e,

conforme o registro do notebook, utilizou CPU. O processo completo de treinamento teve duração aproximada de 4,769 horas.

Durante o treinamento, o modelo apresentou evolução das perdas de caixa (box loss), classificação (classification loss) e distribuição focal (DFL loss). Na décima época, os valores registrados foram 1,694 para box loss, 1,861 para classification loss e 1,411 para DFL loss. Na validação final, realizada com os pesos armazenados em best.pt, foram obtidos os resultados apresentados na **Tabela 1.**

2.3 Métricas de avaliação

A avaliação foi realizada utilizando precisão (Precision), revocação (Recall), mAP@50 e mAP@50–95. A precisão representa a proporção de detecções classificadas corretamente entre as detecções realizadas, enquanto o recall indica a capacidade do modelo de recuperar as ocorrências presentes nas imagens. O mAP é uma medida agregada de desempenho da detecção, sendo calculado para diferentes critérios de interseção sobre união (IoU).

Conj.	Imag.	Instân.	Precision	Recall
Validação	456	683	0,674	0,470
Validação	456	683	mAP@50 = 0,497	mAP@50–95 = 0,247

Tabela 1: Resultados globais obtidos na validação final do YOLOv8n.

3 Resultados e Discussões

A validação final apresentou precisão de 0,674 e recall de 0,470. O valor de mAP@50 foi 0,497, enquanto o mAP@50–95 foi 0,247. Esses valores mostram que o modelo conseguiu realizar detecções de diferentes classes de defeitos, mas ainda apresenta margem significativa para melhoria, principalmente na localização mais precisa das caixas e na recuperação de todas as ocorrências.

O desempenho variou consideravelmente entre as classes. A classe 3_yueyawan apresentou mAP@50 de 0,952 e recall igual a 1,000, enquanto a classe 1_chongkong apresentou mAP@50 de 0,917 e recall de 0,939. Por outro lado, classes com poucos exemplos apresentaram desempenho inferior. A classe 10_yaozhe possuía apenas 4 imagens e 6 instâncias na validação e apresentou recall igual a zero e mAP@50 de 0,0109. A classe 9_zhehen, com 11 imagens e 19 instâncias, também apresentou recall igual a zero. Esses resultados indicam a influência da quantidade e da distribuição das amostras sobre o treinamento.

A classe 7_yiwu também apresentou desempenho reduzido, com precisão de 0,423, *recall* de 0,129, mAP@50 de 0,268 e mAP@50–95 de 0,0896. Em contraste, classes com maior quantidade de exemplos, como 6_siban, 2_hanfeng e 1_chongkong, apresentaram resultados mais consistentes. Portanto, os resultados sugerem que o aumento e o balanceamento das amostras das classes minoritárias podem contribuir para melhorar a capacidade de generalização do modelo.

Classe	Imag.	Inst.	P	R	mAP50	mAP50–95
6_siban	148	175	0.709	0.432	0.517	0.241
4_shuiban	52	63	0.688	0.571	0.603	0.352
5_youban	34	68	0.547	0.544	0.520	0.261
7_yiwu	49	85	0.423	0.129	0.268	0.090
1_chongkong	66	66	0.888	0.939	0.917	0.462
2_hanfeng	97	98	0.703	0.867	0.783	0.347
10_yaozhe	4	6	1.000	0.000	0.011	0.004
3_yueyawan	50	50	0.880	1.000	0.952	0.619
9_zhehen	11	19	1.000	0.000	0.362	0.120
10_yaozhed	34	34	0.282	0.529	0.362	0.125
8_yahen	9	19	0.296	0.158	0.175	0.094

Tabela 2: Métricas por classe obtidas na validação final.

O recall global de 0,470 indica que mais da metade dos defeitos não foi detectada. Em inspeção industrial, isso equivale a falsos negativos, ou seja, peças defeituosas que seriam aprovadas, o que é mais crítico do que falsos positivos. As principais causas prováveis são o treinamento curto (10 épocas, ainda sem convergência, pois as perdas continuavam altas), o desbalanceamento de classes (*10_yaozhe* e *9_zhehen* com recall zero), o uso do modelo de menor capacidade (YOLOv8n) e defeitos pequenos ou de baixo contraste. Seriam necessárias mais épocas, aumento de dados e ajuste do limiar de confiança para priorizar o recall.

Como o conjunto de validação foi usado para avaliar e para selecionar o melhor modelo (best.pt), as métricas podem estar otimistas. Não houve um conjunto de teste independente (por exemplo, 70/15/15), o que limita a avaliação da generalização. A distribuição das

classes pode ser observada na **Imagem 1** e **Imagem 2**, com as Matrizes de Confusão.

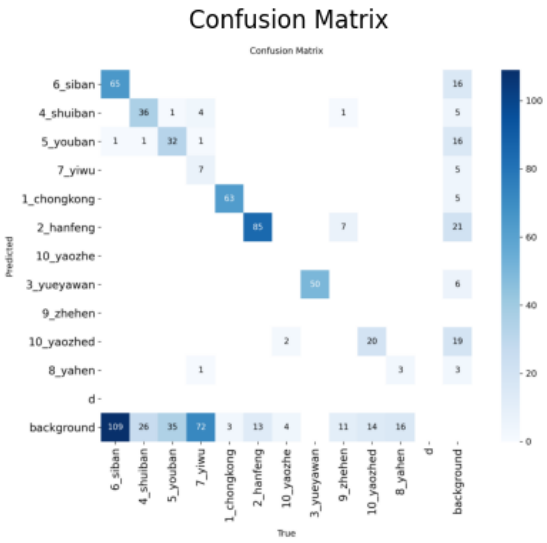


Imagem 1 – *Matriz de Confusão*

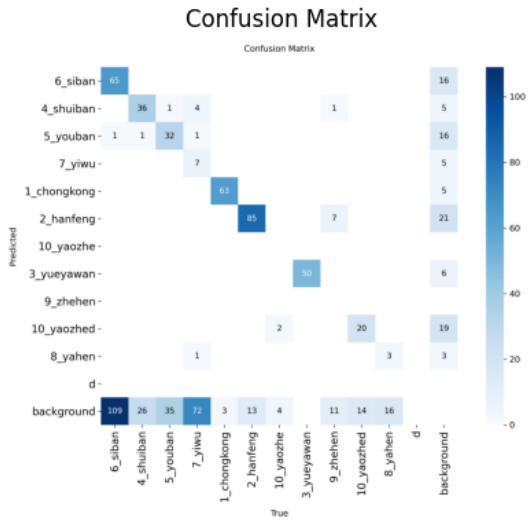


Imagem 2 – *Matriz de Confusão Normalizada*

3.1 Teste de Inferência

Após o treinamento, o arquivo best.pt foi utilizado para realizar inferência em uma imagem externa obtida por URL. O resultado registrado no notebook indicou uma detecção da classe 2_hanfeng. A inferência foi executada em uma imagem redimensionada para 288 × 640 durante o processamento, com tempo aproximado de 196,0 ms para a inferência, além de 7,1 ms de pré-processamento e 30,4 ms de pós-processamento.

O teste também confirmou a existência de uma bounding box, com coordenadas aproximadas [59,23; 158,50; 392,12; 182,63] e confiança de 0,5949. O ID de classe retornado foi 5, correspondente à classe 2_hanfeng na configuração gerada pelo dataset. Não foram identificadas máscaras de segmentação, pois o experimento foi configurado para detecção por caixas delimitadoras.

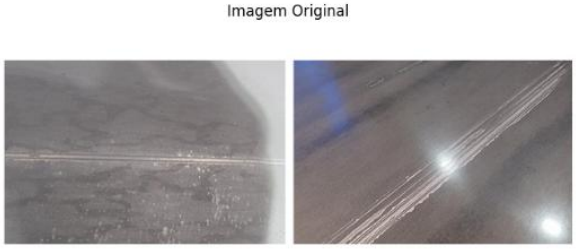


Imagem 3: *Imagem Original.*

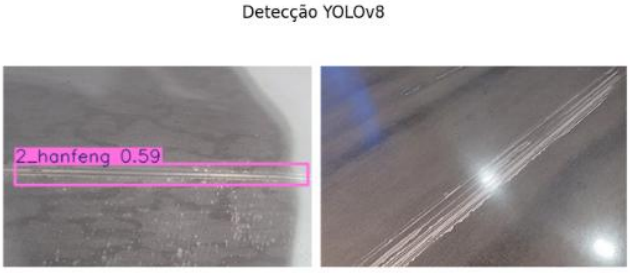


Imagem 4: *Imagem com Detecção Yolov8.*

3.2 Conjunto de dados (12 vs. 11 classes)

O conjunto de dados utilizado neste trabalho apresenta 12 classes de defeitos em peças metálicas. Entretanto, durante a etapa de validação do modelo YOLO, foram identificadas apenas 11 classes. Essa diferença indica que uma das categorias não apresentou ocorrências no subconjunto de validação ou que pode existir alguma inconsistência na identificação e no mapeamento das classes durante o processamento das anotações.

Essa diferença entre o número de classes disponíveis no conjunto original e aquelas presentes na validação é importante, pois pode influenciar a avaliação do desempenho do modelo. A ausência de exemplos de determinada categoria na validação impossibilita verificar

diretamente sua capacidade de reconhecimento para essa classe específica.

Além disso, a distribuição das imagens e das instâncias entre treinamento e validação deve ser considerada, uma vez que o equilíbrio entre as categorias contribui para uma avaliação mais representativa do modelo. Portanto, a diferença observada entre as 12 classes originais e as 11 classes identificadas na validação deve ser considerada na interpretação dos resultados experimentais.

4 Conclusões

Este trabalho constitui uma prova de conceito da aplicação do YOLOv8n à inspeção de peças metálicas. Os resultados (mAP@50 de 0,497 e recall de 0,470), obtidos apenas em validação, com treinamento curto e classes desbalanceadas, ainda não permitem afirmar que a solução seja robusta o bastante para uso industrial. São necessários mais dados nas classes minoritárias, treinamento mais longo, modelos maiores e avaliação em um conjunto de teste independente.

O experimento demonstrou a possibilidade de utilizar o YOLOv8n para a inspeção visual automatizada de peças metálicas. A metodologia contemplou a conversão das anotações XML para o formato YOLO, a divisão do dataset em treinamento e validação, o treinamento do modelo e a realização de inferência em uma imagem externa. Assim, o experimento indica que o sistema é funcional como prova de conceito, mas ainda requer aprimoramentos antes de ser considerado uma solução robusta para uso industrial.

Como trabalhos futuros, recomenda-se ampliar o número de imagens das classes minoritárias, revisar a qualidade e consistência das anotações, aumentar o número de épocas, avaliar modelos YOLO de maior capacidade e comparar diferentes configurações de treinamento. Também é recomendável realizar uma avaliação com um conjunto de teste independente, permitindo medir de forma mais rigorosa a capacidade de generalização do sistema.

Referências

GONZALEZ, Rafael C.; WOODS, Richard E. Processamento Digital de Imagens. 3. ed. São Paulo: Pearson Prentice Hall, 2010.

MAIA, Helton. Visão Computacional. Disponível em: https://heltonmaia.com/computervision/intro.html. Acesso em: 22 jul. 2025.

ULTRALYTICS. Ultralytics YOLO. Documentação e biblioteca para detecção de objetos. Disponível em: https://docs.ultralytics.com/. Acesso em: 3 out. 2026.

XIE, X. A review of recent advances in surface defect detection using texture analysis techniques. Image and Vision Computing, v. 26, n. 6, p. 850–863, 2008.

LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. Deep learning. Nature, v. 521, p. 436–444, 2015.

REDMON, Joseph et al. You Only Look Once: Unified, Real-Time Object Detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, p. 779–788.